

GPart: A GNN-Enabled Multilevel Graph Partitioner

Magi Chen and Ting-Chi Wang

Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan

Abstract—This paper introduces GPart, a scalable multilevel framework for graph partitioning that integrates GNN embeddings with efficient coarsening and refinement techniques. On the Titan23 benchmarks, GPart achieves a cut size reduction of 34.13% to 42.92% over METIS and improves cut size by 9.30% on selected DIMACS benchmarks compared to G-kway. Furthermore, experiments on the Titan23 benchmarks show that GPart reduces normalized memory usage by 24.6x compared to GAP and 12.4x compared to GenPart. Unlike existing GNN-based methods, which require large hidden layers and substantial memory, GPart’s multilevel architecture reduces hidden layer sizes, significantly optimizing memory efficiency.

Index Terms—graph neural network, graph partitioning

I. INTRODUCTION

Graph partitioning is a fundamental problem with extensive applications across various domains, including VLSI design, social network analysis, and scientific computing. Effective partitioning optimizes the allocation of graph components to different partitions, which reduces communication overhead and enhances computational efficiency. However, graph partitioning is an NP-hard problem, making it challenging to achieve both high-quality cuts and balanced partitions, especially as graph sizes continue to grow. In the VLSI domain, optimized graph partitioning plays a crucial role in dividing complex circuit designs into manageable subcomponents, facilitating efficient and parallel processing.

The multilevel approach has been successful in graph partitioning by decomposing the problem into coarsening, initial partitioning, and uncoarsening phases. METIS [2], a notable method, introduced the heavy edge matching (HEM) technique for efficient graph coarsening. To handle larger graphs, GPU acceleration has been leveraged to enhance scalability. GPU-based methods like G-kway [5] and [7] introduce parallelized coarsening mechanisms, demonstrating that GPUs can scale graph partitioning without compromising quality.

Spectral techniques have also been effective in capturing global structural properties for graph partitioning. Methods utilizing the Fiedler vector [16] and higher-order eigenvectors [15] have shown improved partitioning quality, including multi-way partitioning. For hypergraph partitioning, hypergraphs are converted into clique expansion graphs to enable spectral methods [14], [17], [18]. Recent works like SpecPart [17] and K-SpecPart [18] incorporate supervised hints to optimize vertex embeddings. However, spectral methods face scalability challenges due to increasing computational complexity with the number of eigenvectors required, especially for multi-way partitioning tasks.

To address these challenges, multilevel spectral coarsening strategies have been explored. GraphZoom [19] introduced approximate calculations to reduce computational costs while preserving key spectral properties. MedPart [20] adapted similar techniques for 2-way hypergraph partitioning. Nonetheless, these methods primarily focus on approximating the Fiedler vector and do not effectively extend to higher-order eigenvectors, limiting their applicability for k -way partitioning when $k \geq 3$.

Graph Neural Network (GNN)-based techniques have emerged as another approach. GAP [1], [6] uses an autoencoder-based architecture with GraphSAGE [8] layers and a normalized cut loss function for unsupervised learning, achieving competitive partitioning quality.

GenPart [9] extends this framework to hypergraphs. Despite these advancements, GNN-based methods struggle with high computational complexity and memory usage, especially as the depth and width of hidden layers increase, posing difficulties in scaling to larger graphs.

To overcome these limitations, we propose **GPart**, a novel multilevel graph partitioner that integrates GNN embeddings with efficient coarsening and refinement techniques. GPart builds upon traditional methods and introduces innovative strategies to enhance performance while reducing computational and memory costs. Our contributions are summarized as follows:

- 1) We introduce GPart, the first multilevel graph partitioning framework that integrates learnable coarsening guided by GNN-generated node embeddings. Through *unsupervised* training with a carefully designed loss function, GPart captures essential structural properties by approximating both the Fiedler vector and higher-order eigenvectors. Its multilevel approach significantly reduces memory usage compared to existing GNN-based partitioners like GAP and GenPart, making it suitable for large-scale graphs.
- 2) GPart introduces a centroid-guided, gain-based refinement strategy during the uncoarsening phase. By leveraging learned node embeddings to compute weighted centroids for each partition, GPart identifies candidate nodes whose current assignments differ from their nearest centroids in the embedding space. It then selectively reassigns high-gain candidates—those offering the greatest improvement in cut size—while limiting the number of moves per level to ensure computational efficiency and maintain balance constraints.
- 3) Experimental evaluations on the Titan23 [13] and DIMACS [11] benchmarks demonstrate GPart’s capacity to achieve notable improvements in cut size and competitive execution times over METIS. Additionally, experiments on the Titan23 benchmarks demonstrate that GPart reduces normalized memory usage by 24.6x compared to GAP and 12.4x compared to GenPart.

The remainder of this paper is organized as follows: Section II illustrates the problem formulation for graph partitioning. Sections III and IV describe the proposed approach and details. Section V presents our experimental results and analyzes the improvements achieved. Finally, Section VI concludes the paper.

II. PROBLEM FORMULATION

A graph $G(V, E)$ consists of a vertex set V and an edge set E , where each edge $e \in E$ connects a pair of vertices in V . Each vertex v is assigned a weight w_v , and each edge e carries a weight w_e .

Given a graph $G(V, E)$, a positive integer $k \geq 2$, and a balance factor $0 < \varepsilon \ll 1$, the k -way graph partitioning problem is to partition the vertex set V into a collection of k disjoint subsets $P = \{P_0, P_1, \dots, P_{k-1}\}$, where each P_i represents a partition. The total weight of the vertex set is denoted by $W = \sum_{v \in V} w_v$. The goal is to minimize the cut size, defined as:

$$\text{cutsizes}(P) = \sum_{e \in \{(u,v) \in E \mid u \in P_i, v \in P_j, \text{ and } P_i \neq P_j\}} w_e,$$

subject to the following balance constraint:

$$\sum_{v \in P_i} w_v \leq (1 + \varepsilon) \frac{W}{k}, \quad \text{for } 0 \leq i \leq k - 1.$$

To simplify the presentation and analysis, this paper assumes uniform weights for vertices and edges, i.e., $w_v = 1$ for all $v \in V$ and $w_e = 1$ for all $e \in E$.

III. OVERVIEW OF GPART

GPart is an efficient framework for graph partitioning, incorporating modules for embedding, coarsening, uncoarsening (projection), and refinement (See Figure 1). Unlike end-to-end GNN-based methods, GPart employs a modular approach with separate training and inference processes, enhancing both memory efficiency and partitioning quality.

In the **Training Phase**, GPart employs multilevel coarsening with GNN-based embeddings to guide the merging of nodes and minimize cut size. Afterward, embeddings are compressed and refined, preparing them for partitioning.

In the **Inference Phase**, the trained embedding and coarsening layers are reused, with an additional centroid-guided, gain-based refinement process to further improve cut quality while keeping the balance constraint. This approach identifies nodes with high potential for enhancement, enabling selective adjustments to the partitions based on centroid proximity. The inference process iteratively refines partitions until the original graph granularity is restored, yielding the final partitioning solution.

IV. DETAILS OF GPART

In this section, we describe the components of GPart's training and inference frameworks, providing a detailed breakdown of each module. Additionally, we explain the loss functions utilized during training to enhance model performance.

A. Details of Training Framework

1) *Embedding Layers*: The embedding layers in GPart utilize a three-layer GraphSAGE architecture [8] to generate high-dimensional node embeddings, which serve as the foundation for the subsequent stages in the partitioning process. GraphSAGE is specifically chosen for its inductive learning capabilities, enabling the model to generalize effectively across different graph structures by aggregating information from node neighborhoods.

The initial feature of each node in the input graph is derived from a unique sequence ID (0,1,2,...), assigned to each node, which is then standardized by subtracting the mean and dividing by the standard deviation. These standardized IDs serve as the input to the first layer of the embedding layers.

The embedding layers are structured as three consecutive GraphSAGE layers, with input and output dimensions of $1 \rightarrow 16$, $16 \rightarrow 16$, and $16 \rightarrow 16$, respectively. Each layer employs a mean aggregation function, allowing the model to capture increasingly higher-order neighborhood information with each layer. This progressively enriched feature representation enables the embeddings to reflect both local and broader structural contexts within the graph. The feature update equation for each layer t is as follows:

$$f_i^{(t)} = \sigma \left(W^{(t)} \cdot \text{AGGREGATE} \left(\{f_j^{(t-1)}, \forall j \in \mathcal{N}(i)\} \right) + W_{\text{self}}^{(t)} \cdot f_i^{(t-1)} \right)$$

Here, $f_i^{(t)}$ represents the embedding of node i at layer t , $\mathcal{N}(i)$ denotes the neighbors of node i , and σ is a nonlinear activation function (in this work, tanh). The parameters $W^{(t)}$ and $W_{\text{self}}^{(t)}$ are learned

weight matrices for neighborhood and self-feature transformation, respectively. This update allows each node to incorporate information from its immediate neighbors, progressively capturing higher-order neighborhood structures as the layers deepen.

The three-layer GraphSAGE model progressively enriches each node's feature representation by capturing higher-order neighborhood information while maintaining a consistent 16-dimensional hidden layer size. The final layer normalizes embeddings to unit length, aligning them with the unit-norm property of eigenvectors. This normalization encourages the embeddings to capture structural information akin to Laplacian eigenvectors, enhancing compatibility with spectral-based clustering and coarsening techniques.

2) *Coarsening Module*: The Coarsening Module in GPart employs a multilevel coarsening approach, allowing a reduction in GNN layer sizes and thus significantly decreasing memory usage. By iteratively simplifying the graph, this approach enables efficient computation while preserving essential structural properties, making it well-suited for large-scale graph partitioning tasks.

GPart's coarsening process leverages Graclus [3], a GPU-accelerated clustering algorithm that performs greedy matching to group nodes based on edge weights. Graclus aggregates neighboring nodes with strong connections to form compact, representative clusters. Typically, each level of coarsening reduces the number of nodes by approximately half, enabling an exponential decrease in graph size across multiple levels.

In GPart, the coarsening process is further refined with a similarity-based weighting mechanism, where edge weights are adjusted according to node embedding similarity. Specifically, the weight w_{uv} of an edge between nodes u and v is modified as follows:

$$w_{uv} = \text{sigmoid}(-\|\mathbf{x}_u - \mathbf{x}_v\|_2) \times \text{original_weight}_{uv}$$

where $\mathbf{x}_u$ and $\mathbf{x}_v$ are the embeddings of nodes u and v , and $\|\mathbf{x}_u - \mathbf{x}_v\|_2$ represents their Euclidean distance. This distance-based similarity score, modified by a sigmoid function, enhances edge weights for closely connected nodes, guiding the matching process toward forming clusters that are both structurally cohesive and feature-similar.

As nodes are merged into clusters, their inter-cluster edge weights are aggregated to reflect the combined connectivity strength, while each cluster's size is tracked to support the initial partitioning phase. This aggregation results in a coarsened graph representation with weighted edges and node sizes, ensuring respect for cluster sizes during partitioning.

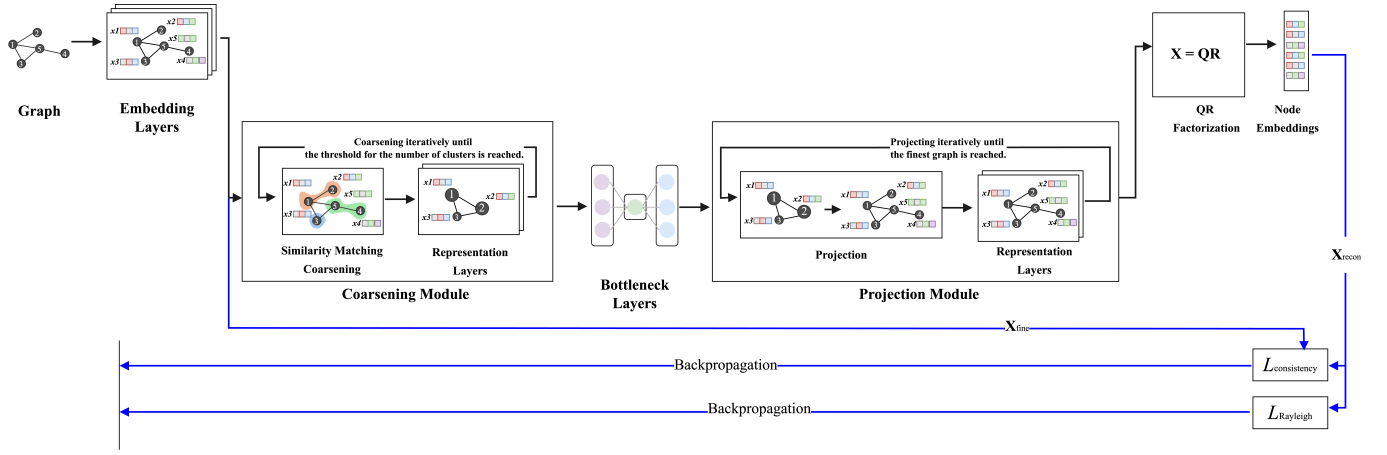
Within the Coarsening Module, **Representation Layers** are applied to refine embeddings at each coarsening level. These layers consist of two consecutive GraphSAGE layers with dimensions $16 \rightarrow 16$.

The coarsening process continues until the graph reaches a pre-defined size threshold, typically set to around $\frac{1}{20}$ of the original node count, following the G-kway strategy [5] and facilitating direct comparison with G-kway in later evaluations.

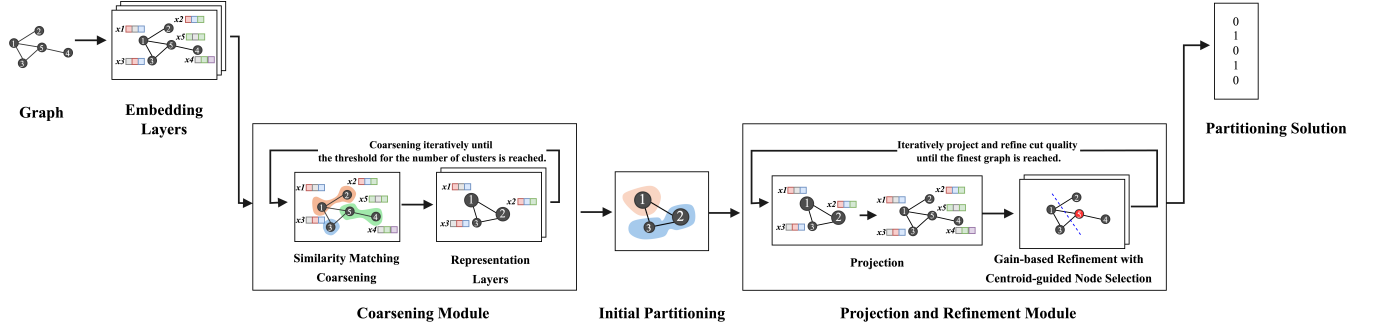
3) *Bottleneck Layers*: The Bottleneck Layers in GPart serve as a compression and refinement mechanism for node embeddings, applied exclusively during the training phase.

The Bottleneck Layers are structured with four linear layers, designed with the following dimensional configuration: $16 \rightarrow 8$, $8 \rightarrow 8$, $8 \rightarrow 8$, and $8 \rightarrow 16$. The bottleneck effect of these layers plays a crucial role in helping the coarsening module learn more focused and meaningful embeddings.

4) *Projection Module*: The Projection Module in GPart is responsible for reconstructing the original graph structure from the



(a) Training Framework of GPart.



(b) Inference Framework of GPart.

Fig. 1: The training and inference frameworks of GPart, illustrated with a 2-way partitioning example. The training framework includes embedding layers, coarsening module, bottleneck layers, projection module and QR factorization to generate node embeddings. The blue lines represent the backpropagation flow, applying consistency and Rayleigh losses to optimize embeddings through coarsening and projection modules. In the inference framework, the trained embedding layers and coarsening module are reused, followed by initial partitioning, iterative projection and refinement to produce the final partitioning solution.

coarsened representation, enabling refined and accurate final partitioning. This module utilizes a GPU-accelerated uncoarsening process to iteratively reverse the coarsening steps, projecting embeddings from the simplified graph back to finer levels while maintaining computational efficiency. During the training phase, the Projection Module focuses solely on restoring the graph’s structure and refining embeddings, without performing any cut quality refinement. The refinement of cut quality is reserved exclusively for the inference phase.

The projection process begins with the embeddings from the deepest level of the coarsening hierarchy. Using the cluster mappings recorded during coarsening, the Projection Module “unpools” the embeddings back to finer graph levels. Mathematically, given the embeddings $\mathbf{X}^{(l)}$ at coarsening level l , the unpooling process maps these embeddings to the next finer level $l - 1$ based on cluster information. Let $\mathbf{M}^{(l-1)}$ be the mapping matrix from clusters at level l to nodes at level $l - 1$. The embeddings at level $l - 1$ are obtained as:

$$\mathbf{X}^{(l-1)} = \mathbf{M}^{(l-1)} \mathbf{X}^{(l)}$$

where $\mathbf{M}^{(l-1)}$ is a mapping matrix that associates each node in the finer graph at level $l - 1$ with the embedding of its corresponding cluster from the coarser graph at level l .

To refine the embeddings during this projection process when training, the Projection Module employs **Representation Layers**,

consisting of two GraphSAGE layers with dimensions $16 \rightarrow 16$ each.

This iterative projection process continues until the finest granularity of the graph is reached, effectively restoring the input graph structure and embedding resolution. QR factorization is then applied to the embeddings to enforce orthogonality.

B. Loss Functions

After the QR factorization, GPart employs Rayleigh Quotient-based loss functions during training to ensure that the node embeddings capture essential spectral properties of the graph. These loss functions guide the learning process, promoting effective encoding of both local structural details and broader spectral characteristics crucial for graph partitioning.

1) *Spectral Consistency Loss*: The **Spectral Consistency Loss** is designed to preserve the spectral properties of the graph in the node embeddings produced by the embedding layers (referred to as the fine graph) and the reconstructed graph after coarsening and uncoarsening. Since the Laplacian matrix $\mathbf{L}$ of the fine graph is identical to that of the reconstructed graph, this loss compares the Rayleigh quotients of the node embeddings from both graphs using the same Laplacian matrix.

The Rayleigh quotient for a node embedding vector $\mathbf{x}$ with respect to a Laplacian matrix $\mathbf{L}$ is defined as:

$$R(\mathbf{x}, \mathbf{L}) = \frac{\mathbf{x}^\top \mathbf{L} \mathbf{x}}{\mathbf{x}^\top \mathbf{x}}. \quad (1)$$

Here, $\mathbf{x}^\top$ denotes the transpose of $\mathbf{x}$.

Let $\mathbf{x}_{\text{fine}}$ and $\mathbf{x}_{\text{recon}}$ represent the node embeddings of the fine and reconstructed graphs, respectively. The Spectral Consistency Loss is then defined as:

$$\mathcal{L}_{\text{consistency}} = \text{MSE}(R(\mathbf{x}_{\text{fine}}, \mathbf{L}), R(\mathbf{x}_{\text{recon}}, \mathbf{L})), \quad (2)$$

where MSE denotes the Mean Squared Error. This loss function minimizes the discrepancy between the spectral characteristics of the node embeddings from the fine and reconstructed graphs. By aligning their Rayleigh quotients, we ensure that the embeddings preserve important structural information during coarsening and uncoarsening.

2) **Comprehensive Rayleigh Quotient Alignment Loss:** The **Comprehensive Rayleigh Quotient Alignment Loss** is a composite loss comprising several components, each targeting a specific aspect of spectral alignment:

- **Spectral Residual Loss:** This component encourages the embeddings to align with the eigenvectors of the Laplacian matrix $\mathbf{L}$. It minimizes the residual between the Laplacian-transformed embedding $\mathbf{L}\mathbf{x}_{\text{recon}}$ and the embedding scaled by its Rayleigh quotient $R(\mathbf{x}_{\text{recon}}, \mathbf{L}) \cdot \mathbf{x}_{\text{recon}}$. This is based on the eigenvalue equation $\mathbf{L}\mathbf{x} = \lambda\mathbf{x}$, where λ is the eigenvalue. The Spectral Residual Loss is defined as:

$$\mathcal{L}_{\text{residual}} = \text{MSE}(\mathbf{L}\mathbf{x}_{\text{recon}}, R(\mathbf{x}_{\text{recon}}, \mathbf{L}) \cdot \mathbf{x}_{\text{recon}}). \quad (3)$$

By minimizing this residual, we encourage the embeddings to approximate the eigenvectors of $\mathbf{L}$, aligning them with the graph's spectral properties.

- **Fiedler Vector Approximation Loss:** The Fiedler vector corresponds to the eigenvector associated with the second smallest eigenvalue of $\mathbf{L}$ and is crucial for capturing the graph's community structure. By minimizing the mean Rayleigh quotient across all embedding dimensions, we encourage the embeddings to align with the Fiedler vector.

The Fiedler Vector Approximation Loss is defined as:

$$\mathcal{L}_{\text{fiedler}} = \frac{1}{d} \sum_{i=1}^d R(\mathbf{x}_{\text{recon}, i}, \mathbf{L}), \quad (4)$$

where d is the embedding dimension, and $\mathbf{x}_{\text{recon}, i}$ denotes the i -th column of the reconstructed embeddings.

- **Higher-Order Spectral Loss:** This component aims to capture eigenvectors associated with larger eigenvalues (higher-frequency components), enabling the embeddings to encode more complex structural features important for multi-way partitioning. We define a high-pass filtered version of the embeddings:

$$\mathbf{h}_{\text{high}} = \mathbf{x}_{\text{recon}} - \alpha \mathbf{L}\mathbf{x}_{\text{recon}}, \quad (5)$$

where α is a scaling factor that adjusts the influence of the Laplacian term, controlling the emphasis on higher-frequency components.

The Higher-Order Spectral Loss is then defined as:

$$\mathcal{L}_{\text{higher_order}} = \frac{1}{R(\mathbf{h}_{\text{high}}, \mathbf{L})}. \quad (6)$$

By minimizing the inverse of the Rayleigh quotient of $\mathbf{h}_{\text{high}}$, we encourage these high-frequency components to align with higher-order eigenvectors of $\mathbf{L}$.

The combined Comprehensive Rayleigh Quotient Alignment Loss is expressed as:

$$\mathcal{L}_{\text{Rayleigh}} = \mathcal{L}_{\text{residual}} + \lambda_1 \mathcal{L}_{\text{fiedler}} + \lambda_2 \mathcal{L}_{\text{higher_order}}, \quad (7)$$

where λ_1 and λ_2 are weighting factors that balance the contributions of each component.

3) **Total Loss Function:** In summary, the total loss function used in GPart combines the Spectral Consistency Loss and the Comprehensive Rayleigh Quotient Alignment Loss:

$$\mathcal{L} = \mathcal{L}_{\text{Rayleigh}} + \mathcal{L}_{\text{consistency}}. \quad (8)$$

Since the Laplacian matrix $\mathbf{L}$ can be computed from the input graph, no labeled data is required, making this an unsupervised approach for graph partitioning tasks. By carefully designing these loss functions, we guide the embeddings to capture both global and local structural properties essential for effective graph partitioning.

C. Details of Inference Framework

The inference framework in GPart is designed to leverage the components trained during the learning phase, specifically reusing the embedding layers and coarsening module without requiring further training. The inference pipeline is composed of embedding layers, coarsening module, an initial partitioning step and a projection and refinement phase, with each stage meticulously structured to optimize partitioning quality and balance across partitions.

1) **Initial Partitioning:** The primary objective of the initial partitioning step is to establish a foundational partitioning scheme for the coarsened graph, which then serves as a basis for further refinement. Given the reduced size of the most coarsened graph, GPart utilizes the single-threaded METIS algorithm to perform this initial partitioning.

2) **Projection and Refinement Module:** In the projection and refinement phase, GPart reconstructs the input graph by progressively uncoarsening it from the coarsest level back to its original, finest granularity. At each uncoarsening level, GPart performs a refinement step to enhance partitioning quality by optimizing the cut size while ensuring balance across partitions.

Traditional refinement algorithms, such as the Fiduccia-Mattheyses (FM) algorithm [4], evaluate and potentially reassign every node based on computed gains, which is computationally intensive for large-scale graphs. GPart introduces an efficient refinement strategy that reduces computational overhead by leveraging the learned node embeddings and a centroid-based method.

Specifically, GPart calculates **weighted centroids** for each partition using the node embeddings generated by the coarsening module. These embeddings capture essential structural and spectral properties of the graph, positioning nodes in a high-dimensional space where proximity reflects structural similarity. This makes centroid calculations both meaningful and computationally feasible, especially on GPUs.

The refinement process focuses on nodes whose current partition assignments—determined by the solution from the previous coarser level—differ from the assignments suggested by their nearest centroids in the embedding space. Among these **candidate nodes**, GPart computes the **gain** for each node, representing the potential improvement in cut size if the node were moved to a different partition.

GPart prioritizes moving nodes with the highest gains. After each node movement, it recalculates the gains of affected neighboring nodes to reflect changes in the partitioning state. Throughout this process, GPart continuously tracks and records the partitioning configuration that achieves the best cumulative gain improvement. By the end of the refinement phase, even if subsequent moves do not yield better results, GPart selects the partitioning configuration with the highest recorded gain improvement among all moves performed.

To maintain computational efficiency, GPart restricts the refinement process to a maximum of **50** node moves per uncoarsening level, ensuring that only the most impactful node reassignments are performed while adhering to balance constraints.

This targeted gain-based refinement approach, combined with embedding-based centroid calculations, enhances both the computational efficiency and effectiveness of the partitioning process.

V. EXPERIMENTAL VALIDATION

A. Experimental Setup

The experimental validation of GPart was conducted using a framework developed with Python, PyTorch, and the PyTorch Geometric library. The experiments were run on a machine equipped with an Intel Core i7-13700 CPU, 128GB of memory, and an Nvidia RTX 4090 GPU with 24GB of VRAM.

1) *Dataset*: For training, we generated a dataset of subgraphs derived from large graphs selected from the DIMACS Graph Partitioning Challenge [11]¹. From these base graphs, we randomly extracted 1600 subgraphs, each containing approximately 20,000 nodes.

2) *Training Configuration:* GPart was trained using the Adam optimizer for 1500 epochs. The initial learning rate was set to 0.00005 and was adjusted using a learning rate scheduler with a step size of 50 and a decay factor of 0.2.

3) *Loss Function Parameters:* The loss function in GPart incorporates weighted components to balance various optimization objectives. The weights for the loss function components were set to $\lambda_1 = 1$ and $\lambda_2 = 0.01$, ensuring that the $L_{fiedler}$ and L_{high_order} terms are appropriately balanced during training. Additionally, a scaling factor $\alpha = 1$ was used to regulate the high-frequency emphasis within the loss.

4) *Experimental Design*: The experimental design includes the following analyses:

- 1) **Cut Size and Runtime Comparison:** To assess partitioning quality and computational efficiency, we compare GPart with METIS [2] on the Titan23 circuit benchmark suite [13] and on graphs from the DIMACS Graph Partitioning Challenge [11]. Additionally, we include a comparison with G-kway [5] on the same DIMACS benchmarks used in its evaluation.
- 2) **Memory Usage Analysis:** We evaluate the memory efficiency of GPart on the Titan23 benchmarks by comparing its memory usage with that of GAP [1] and GenPart [9].

It is important to note that Titan23 is a circuit benchmark suite represented as hypergraphs. For compatibility with graph partitioning algorithms, we convert the Titan23 benchmarks into clique expansion graphs. In this conversion, hyperedges with degrees exceeding 1000 are removed to focus the analysis on more manageable hyperedge sizes. Table VI summarizes the statistics of the graphs converted from the Titan23 benchmarks.

B. Cut Size and Runtime Comparison

1) *Results on Titan23 Benchmarks:* In this section, we compare the performance of GPart with METIS in terms of cut size and runtime efficiency on the Titan23 circuit benchmark suite. The experiments were conducted with partition counts $k = 2$ and $k = 3$, and balance factors ε of 3% and 10%.

The results in Table I present the cut size comparison for METIS and GPart at $k = 2$. GPart consistently demonstrates significant

¹The graphs used include *G3_circuit*, *asia.osm*, *delaunay_n15*, *delaunay_n16*, *delaunay_n17*, *delaunay_n18*, *delaunay_19*, *delaunay_20*, *ldoor*, and *NLR*.

TABLE I: Cut size comparison for METIS and GPart on Titan23 benchmarks at $k = 2$ with balance factors ε of 3% and 10%.

	$k = 2, \epsilon = 3\%$			$k = 2, \epsilon = 10\%$		
Benchmark	<i>METIS</i>	<i>GPart</i>	Improvement (%)	<i>METIS</i>	<i>GPart</i>	Improvement (%)
bitcoin_miner	4,173	2,904	30.41%	4,169	2,628	36.96%
bitonic_mesh	142,095	150,766	-6.10%	142,427	140,738	1.19%
cholesky_bdti	37,439	19,066	49.07%	35,755	18,557	48.10%
cholesky_mc	14,882	4,435	70.20%	8,949	4,173	53.37%
dart	44,648	35,857	19.69%	47,938	36,453	23.96%
denoise	1,976	1,518	23.18%	1,785	1,151	35.52%
des90	142,863	77,042	46.07%	142,870	78,970	44.73%
directrf	30,901	8,042	73.97%	31,201	7,715	75.27%
gsm_switch	54,438	8,189	43.20%	20,062	6,370	68.25%
LU230	14,326	47,267	13.00%	54,047	42,828	20.76%
LU_Network	14,883	13,421	9.82%	12,495	7,911	36.69%
mes_noc	81,956	51,047	37.71%	81,966	36,143	55.90%
minres	10,915	7,951	27.16%	11,162	5,426	51.39%
neuron	6,349	2,926	53.91%	6,322	3,084	51.22%
openCV	28,222	20,190	28.46%	29,169	15,139	48.10%
segmentation	1,661	573	65.50%	7,799	501	93.58%
SLAM_spheric	5,291	5,164	2.40%	5,291	5,041	4.73%
sparcT1_core	19,039	15,509	18.54%	20,953	18,968	9.47%
sparcT2_core	27,217	26,976	0.89%	24,869	18,712	24.76%
stap_qrd	31,383	20,882	33.46%	26,679	17,831	33.16%
stereo_vision	1,212	288	76.24%	1,821	289	84.13%
Average Cut Size Improvement			34.13%			42.92%

improvements in cut size reduction across most benchmarks, with an average improvement of 34.13% at $\varepsilon = 3\%$ and 42.92% at $\varepsilon = 10\%$. These results indicate that GPart can achieve smaller cut sizes compared to METIS, particularly under more relaxed balance constraints (10% balance factor). This improvement is especially notable in benchmarks such as *cholesky_mc*, *directrf*, and *segmentation*, where GPart outperforms METIS by over 65%.

Similarly, Table II provides the cut size comparison for $k = 3$. GPart again demonstrates substantial improvements in cut size, with an average improvement of 31.47% at $\varepsilon = 3\%$ and 36.38% at $\varepsilon = 10\%$. GPart consistently outperforms METIS, particularly in graphs such as *openCV* and *segmentation*, where improvements exceed 56%. However, there are a few cases, such as *bitonic_mesh* at $\varepsilon = 10\%$ and *SLAM_spheric* at $\varepsilon = 3\%$, where METIS slightly outperforms GPart. These exceptions highlight that while GPart generally excels in reducing cut size, specific instances exist where traditional algorithms like METIS remain competitive.

In addition to cut size, we evaluated the execution times of GPart and METIS on the Titan23 benchmarks. The results, shown in Table III, demonstrate that GPart achieves superior time efficiency compared to the Python-wrapped METIS implementation [10]. On average, the initial partitioning step in GPart accounts for approximately 43.16% of the total execution time, highlighting its significance in the overall process. This proportion suggests that while GPart manages the coarsening and refinement steps, the initial partitioning phase remains a substantial component of the total execution time.

TABLE II: Cut size comparison for METIS and GPart on Titan23 benchmarks at $k = 3$ with balance factors ε of 3% and 10%.

	$k=3, \epsilon=3\%$		$k=3, \epsilon=10\%$		
Benchmark	METIS	GPart Improvement (%)	METIS	GPart Improvement (%)	
bitcoin_miner	9,650	8,227	10,350	8,250	20.29%
bitonic_mish	230,579	212,809	194,302	200,764	-3.33%
cholesky_bdti	53,503	31,405	53,595	26,452	50.98%
cholesky_mc	21,754	7,835	20,617	7,351	64.34%
dart	66,685	61,514	70,156	50,369	28.20%
denoise	12,742	4,359	9,878	4,253	56.94%
des90	204,411	106,937	201,682	101,921	49.46%
directrf	38,632	23,860	39,245	20,173	48.60%
gsm_switch	26,532	11,829	28,218	12,118	57.06%
LU230	102,335	88,965	95,167	83,638	12.11%
LU_Network	31,830	26,736	29,255	19,858	32.12%
mes_noc	103,146	84,880	111,964	83,234	25.66%
minres	18,736	12,392	17,931	11,426	36.28%
neuron	8,597	5,352	9,331	4,887	47.63%
openCV	133,203	58,412	96,606	50,337	47.89%
segmentation	8,031	1,880	3,798	1,858	51.08%
SLAM_spheric	31,396	32,577	34,734	30,610	11.87%
sparT1_core	37,576	35,529	35,797	26,465	26.07%
sparT2_core	46,023	39,660	45,776	40,966	10.51%
stap_grd	42,891	32,511	41,638	30,808	26.01%
stereo_vision	2,140	1,554	2,499	894	64.23%
Average Cut Size Improvement		31.47%			36.38%

TABLE III: Comparison of execution times for METIS and GPart.

Benchmark	Time (s)	
	METIS	GPart
bitcoin_miner	7.27	5.72
bitonic_mesh	5.90	2.50
cholesky_bdti	3.47	2.45
cholesky_mc	1.41	1.04
dart	11.32	3.34
denoise	10.35	3.81
des90	2.95	1.35
directrf	12.81	5.91
gsm_switch	25.96	10.96
LU230	7.57	6.86
LU_Network	50.95	15.50
mes_noc	33.09	9.58
minres	3.64	1.83
neuron	1.10	1.25
openCV	10.83	6.22
segmentation	5.68	1.79
SLAM_spheric	4.02	1.22
sparcT1_core	7.55	2.44
sparcT2_core	27.30	8.77
stap_grd	2.86	2.47
stereo_vision	2.04	1.12
Average Normalized Time	1.00	0.53

TABLE IV: Cut size comparison for METIS and GPart on graphs from DIMACS Graph Partitioning Challenge with varied partition counts k .

Benchmark	Statistics		$k = 2, \epsilon = 10\%$			$k = 3, \epsilon = 10\%$		
	V	E	METIS	GPart	Improvement (%)	METIS	GPart	Improvement (%)
delaunay_n19	524,288	1,572,823	1,492	1,361	8.78%	2,326	2,284	1.81%
delaunay_n20	1,048,576	3,145,686	2,035	1,849	9.14%	3,342	2,978	10.89%
delaunay_n21	2,097,152	6,291,408	2,807	2,593	7.62%	4,809	4,235	11.94%
delaunay_n22	4,194,304	12,582,869	4,038	3,694	8.52%	6,646	6,013	9.52%
delaunay_n23	8,388,608	25,165,784	5,888	5,234	11.11%	9,544	8,577	10.13%
delaunay_n24	16,777,216	50,331,601	8,568	7,453	13.01%	13,675	12,069	11.74%
ldoor	952,203	22,785,136	24,696	22,265	9.84%	37,583	33,732	10.25%
NLR	4,163,763	12,487,976	4,091	3,805	6.99%	7,072	6,377	9.83%
asia.osm	11,950,757	12,711,603	7	7	0.00%	19	14	26.32%
Average Cut Size Improvement	-	-	-	-	8.33%	-	-	11.38%

2) *Results on DIMACS Benchmarks:* We further assessed GPart’s performance on graphs from the DIMACS Graph Partitioning Challenge, focusing on cut size improvements over METIS across various graph sizes and partition counts ($k = 2$ and $k = 3$) with a balance factor of $\epsilon = 10\%$.

As shown in Table IV, GPart consistently achieves lower cut sizes than METIS across all tested graphs, with average improvements of 8.33% for $k = 2$ and 11.38% for $k = 3$.

While the industrial benchmark instances used in G-kway [5] are unavailable, we compared GPart’s cut performance for $k = 2$ and $\epsilon = 3\%$ with G-kway on four DIMACS graphs reported in G-kway’s publication, as shown in Table V. The results for G-kway in the table are from [5]. On average, GPart achieved a 9.30% improvement in cut size over G-kway.

C. Memory Usage Analysis

In this section, we present a comparative analysis of memory usage among GAP [1], GenPart [9], and GPart within the context of graph partitioning. GAP and GenPart each employ an end-to-end GNN-based framework that requires both an embedding encoder and a partitioning decoder with large hidden layers to maintain high partitioning quality. This end-to-end design inherently demands greater memory during inference due to the increased number of parameters and activations in the larger hidden layers. In contrast, GPart utilizes a multilevel framework with smaller GNN layers, enabling a significant reduction in hidden layer size.

As stated in [9], memory constraints limit the applicability of GenPart to most Titan23 benchmarks. Due to frequent out-of-memory (OOM) issues during execution, GenPart could not be evaluated on large-scale designs and is therefore excluded from direct comparison. In contrast, GAP’s source code and test cases are not publicly available, preventing us from reproducing its model performance. As a result, our comparison is limited to reported memory usage. Since directly measuring the memory consumption of both GAP

TABLE V: Cut size comparison between G-kway and GPart on graphs from the DIMACS Graph Partitioning Challenge, with a partition count of $k = 2$ and a balance factor of $\epsilon = 3\%$.

Benchmark	Statistics		$k = 2, \epsilon = 3\%$		
	V	E	G-kway [5]	GPart	Improvement (%)
delaunay_n24	16,777,216	50,331,601	8,463	7,465	11.79%
ldoor	952,203	22,785,136	25,578	23,932	6.44%
NLR	4,163,763	12,487,976	4,705	3,812	18.98%
asia.osm	11,950,757	12,711,603	7	7	0.00%
Average Cut Size Improvement	-	-	-	-	9.30%

and GenPart on large-scale benchmarks was impractical due to their high resource demands, we estimated their memory usage based on each model’s architecture and parameters.² To validate our estimation method, we compared GPart’s estimated memory usage against actual measurements on the Titan23 benchmarks, observing a deviation of less than 5%.

Our analysis compares GPart’s embedding and representation layers with the encoders used in GAP and GenPart. GPart’s memory usage, however, was measured directly.

Table VI presents the memory usage comparison for GAP, GenPart, and GPart across benchmarks, with GAP and GenPart’s values being estimated, while GPart’s are measured directly. As shown, GPart achieves significant memory savings, requiring on average 24.6x less memory than GAP and 12.4x less than GenPart, highlighting its memory efficiency.

TABLE VI: Memory usage comparison for GAP, GenPart, and GPart.

Benchmark	Statistics		Memory Usage (GB)		
	V	E	GAP	GenPart	GPart
bitcoin_miner	1,089,284	3,662,788	20.61	10.51	0.81
bitonic_mesh	192,064	12,508,583	49.33	24.92	2.07
cholesky_bdti	266,422	4,660,080	19.54	9.89	0.79
cholesky_mc	113,250	2,592,938	10.67	5.39	0.44
dart	202,354	16,465,022	64.63	32.65	2.72
denoise	275,638	22,737,143	89.23	45.07	3.76
des90	111,221	6,752,436	26.68	13.48	1.12
directrf	931,275	19,432,048	80.41	40.68	3.26
gsm_switch	493,260	60,337,200	235.36	118.87	9.93
LU230	574,372	14,618,649	59.74	30.21	2.45
LU_Network	635,456	82,063,167	319.90	161.56	13.50
mes_noc	547,544	60,370,265	235.81	119.10	9.94
minres	261,359	6,495,763	26.58	13.44	1.09
neuron	92,290	2,770,749	11.23	5.68	0.47
openCV	217,453	28,852,387	112.44	56.78	4.75
segmentation	138,295	10,471,126	41.16	20.79	1.73
SLAM_spheric	113,115	5,636,907	22.39	11.31	0.94
sparcT1_core	91,976	16,697,490	64.87	32.76	2.75
sparcT2_core	300,109	46,519,712	180.99	91.40	7.65
stap_grd	240,240	4,071,793	17.12	8.67	0.69
stereo_vision	94,050	3,785,776	15.15	7.65	0.64
Average Normalized Memory Usage	-	-	24.6x	12.4x	1x

VI. CONCLUSION

We introduced **GPart**, a scalable multilevel graph partitioning framework that integrates GNN-generated embeddings with efficient coarsening and refinement techniques. By capturing essential structural properties through unsupervised training, GPart achieves high-quality partitioning results. Additionally, its multilevel approach significantly reduces memory usage compared to existing GNN-based partitioners, making it suitable for large-scale graphs.

Experimental results on the Titan23 and DIMACS benchmarks demonstrate that GPart achieves substantial improvements in cut size over leading methods like METIS and G-kway. These advancements highlight GPart’s effectiveness and scalability in practical graph partitioning applications.

²GAP uses five GraphSAGE layers with a 512-dimensional hidden size and a 64-dimensional output, while GenPart employs a five-layer GraphConv encoder [12] with a 256-dimensional hidden size and a variational layer as the final layer, also producing a 64-dimensional output.

REFERENCES

- [1] A. Nazi, W. Hang, A. Goldie, S. Ravi, and A. Mirhoseini, "A Deep Learning Framework for Graph Partitioning," in *Proceedings of International Conference on Learning Representations*, 2019.
- [2] G. Karypis and V. Kumar, "A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.
- [3] B. Fagginger Auer and R. Bisseling, "A GPU Algorithm for Greedy Graph Matching," *Facing the Multicore-Challenge*, vol. 2, pp. 108–119, 2012.
- [4] C. M. Fiduccia and R. M. Mattheyses, "A Linear-Time Heuristic for Improving Network Partitions," in *Proceedings of Design Automation Conference*, pp. 175–181, 1982.
- [5] W. L. Lee, D. L. Lin, T. W. Huang, S. Jiang, T. Y. Ho, Y. Lin, and B. Yu, "G-kway: Multilevel GPU-Accelerated k-way Graph Partitioner," in *Proceedings of Design Automation Conference*, 2024.
- [6] A. Nazi, W. Hang, A. Goldie, S. Ravi, and A. Mirhoseini, "GAP: Generalizable Approximate Graph Partitioning Framework," 2019, arXiv:1903.00614.
- [7] B. Goodarzi, F. Khorasani, V. Sarkar, and D. Goswami, "High Performance Multilevel Graph Partitioning on GPU," in *Proceedings of International Conference on High Performance Computing & Simulation*, pp. 769–778, 2019.
- [8] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Proceedings of Neural Information Processing Systems*, 2017.
- [9] M. Chen and T. C. Wang, "A Hypergraph Partitioner Utilizing a Novel Graph Generative Model," in *Proceedings of International Conference on Computer-Aided Design*, 2024.
- [10] A. L. Bruggeman and I. van Gent, "metis-python," [Online]. Available: <https://pypi.org/project/metis/>.
- [11] D. A. Bader, H. Meyerhenke, P. Sanders, and D. Wagner, "Graph Partitioning and Graph Clustering," in *Proceedings of DIMACS Implementation Challenge*, 2012.
- [12] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe, "Weisfeiler and Leman Go Neural: Higher-order Graph Neural Networks," in *Proceedings of AAAI Conference on Artificial Intelligence*, 2018.
- [13] K. E. Murray, S. Whitty, S. Liu, J. Luu, and V. Betz, "Titan: Enabling Large and Complex Benchmarks in Academic CAD," in *Proceedings of International Conference on Field Programmable Logic and Applications*, 2013, pp. 1–8.
- [14] L. W. Hagen and A. B. Kahng, "New Spectral Methods for Ratio Cut Partitioning and Clustering," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 11, 1992, pp. 1074–1085.
- [15] C. J. Alpert and S.-Z. Yao, "Spectral Partitioning: The More Eigenvectors, The Better," in *Proceedings of Design Automation Conference*, 1995, pp. 195–200.
- [16] M. Fiedler, "A Property of Eigenvectors of Nonnegative Symmetric Matrices and its Application to Graph Theory," *Czechoslovak Mathematical Journal*, vol. 25, no. 4, pp. 619–633, 1975.
- [17] I. Bustany, A. B. Kahng, I. Koutis, B. Pramanik, and Z. Wang, "SpecPart: A Supervised Spectral Framework for Hypergraph Partitioning Solution Improvement," in *Proceedings of International Conference on Computer-Aided Design*, 2022.
- [18] I. Bustany, A. B. Kahng, I. Koutis, B. Pramanik, and Z. Wang, "K-SpecPart: Supervised Embedding Algorithms and Cut Overlay for Improved Hypergraph Partitioning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 4, pp. 1232–1245, 2024.
- [19] C. Deng, Z. Zhao, Y. Wang, Z. Zhang, and Z. Feng, "GraphZoom: A Multi-level Spectral Approach for Accurate and Scalable Graph Embedding," in *Proceedings of International Conference on Learning Representations*, 2020.
- [20] R. Liang, A. Agnesina, and H. Ren, "MedPart: A Multi-Level Evolutionary Differentiable Hypergraph Partitioner," in *Proceedings of International Symposium on Physical Design*, 2024, pp. 3–11.
- [21] G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar, "Multilevel Hypergraph Partitioning: Application In VLSI Domain," in *Proceedings of Design Automation Conference*, 1997, pp. 526–529.